OWASP STUDENT CHAPTER • CYBERLAB PLATFORM

OWASP CTF PLATFORM

Backend Feature & API Specification

Frontend Design & Development Reference

DOCUMENT CLASSIFICATION:	Engineering Specification & UI Architecture Reference
TARGET INTERFACE SYSTEM:	Google Stitch / Modern CTF Frontend UI
IMPLEMENTATION SCOPE:	Phases 1–17 Complete (233/233 Automated Tests Passing)
BACKEND ARCHITECTURE:	FastAPI, SQLAlchemy 2.0, PostgreSQL (Supabase), Docker Engine
AUTHOR & GOVERNANCE:	College OWASP Student Chapter Core Development Team
PUBLICATION DATE:	September 2026 • Version 1.0 Production Release

PRIMARY DIRECTIVE FOR GOOGLE STITCH & UI ENGINEERS:
This document is derived directly from the live, fully implemented backend codebase. All endpoints, database fields, roles, constraints, scoring rules, and data structures are production-authoritative. The frontend must consume existing FastAPI endpoints without duplicating scoring calculations or inventing unauthorized interfaces.

TABLE OF CONTENTS

1. Backend Project Structure
2. Database Model Overview
3. Authentication & Authorization
4. API Endpoint Reference
5. Challenge System
6. Scoring System
7. Leaderboards
8. Teams
9. Events & Tournaments
10. Profiles & Activity
11. Badges & Achievements
12. Discussions & Forum
13. Notifications
14. Admin Analytics
15. Security & Anti-Cheat
16. Live Challenge System
17. Security & Deployment
18. Required Frontend Pages
19. Frontend Page → Backend API Mapping
20. Reusable Frontend Components
21. Important User Flows
22. Required UI States
23. Role Permission Matrix
24. Stitch Frontend Design Context
25. Backend Implementation Status

Specification Reference Document for Google Stitch & Frontend Architecture Authoritative, non-invented documentation derived directly from the active, fully-implemented backend codebase (Phases 1–17).

---

1. Backend Project Structure

The CTF Platform backend is constructed with Python 3.14+, FastAPI, SQLAlchemy 2.0+, and Alembic. The tree below reflects the exact implemented repository structure:

```
backend/
├── alembic/                            # Database migration framework
│   ├── env.py                          # Alembic runtime environment, URL resolver, and met
    adata binding
│   ├── README
│   ├── script.py.mako
│   └── versions/                       # 14 Sequential database migration revisions
│       ├── 0001_initial_database_foundation.py     # Initial core users, categories, challenge
    s schema
│       ├── 0002_challenge_management.py          # Challenge files, writeups, metadata extens
    ions
│       ├── 0003_solves_and_submissions.py        # Submissions audit log and solves unique le
    dger
│       ├── 0004_leaderboard_indexes.py           # Composite indexes for high-speed rank quer
    ies
│       ├── 0005_hints_and_writeups.py            # Hint penalty tracking, hint_usage table, w
    riteups
│       ├── 0006_teams_and_members.py             # Teams, team membership, captaincy, invite
    codes
│       ├── 0007_events_and_participation.py      # CTF events, event challenges, team/solo re
    gistration
│       ├── 0008_user_profiles_and_activity.py    # User profiles, bio, college, user_activiti
    es feed
│       ├── 0009_badges_and_achievements.py       # Achievement badges, user badges, 8 seeded
    badges
│       ├── 0010_discussions_and_comments.py      # Discussion topics, nested comment threads
│       ├── 0011_notifications.py                 # User notification center, unread counters
│       ├── 0012_anti_cheat_and_security_events.py # Security incidents, anti-cheat detection,
    audit logs
│       ├── 0013_live_challenge_instances.py      # Live Docker challenge instances, container
     leases
│       └── 0014_user_moderation_fields.py        # Account moderation fields (is_banned, is_m
    uted)
├── alembic.ini                         # Alembic configuration file
├── app/                                # Core FastAPI application package
│   ├── __init__.py
│   ├── config.py                       # Pydantic BaseSettings: environment, JWT, CORS, lim
    its, DB URL
│   ├── database.py                     # SQLAlchemy engine, session maker, connection pool,
     protocol parser
│   ├── init_db.py                      # Standalone programmatic database initialization
│   ├── main.py                         # Application entrypoint, CORS, security headers, ro
    ute mounting
│   ├── core/
│   │   ├── __init__.py
│   │   └── security.py                 # Bcrypt password hashing, verification, JWT creatio
    n/decoding
│   ├── api/                            # 16 Specialized FastAPI Route Modules
│   │   ├── __init__.py
│   │   ├── admin_users.py              # Admin user management, role elevation, banning, mu
    ting
│   │   ├── analytics.py                # Platform, challenge, event, team, and security ana
    lytics
```

```
|   |   ├── anti_cheat.py                # Incident triage, reviewer decisions, user security
    risk scores
|   |   ├── auth.py                      # Registration, login, JWT issuance, profile probe,
    RBAC test
|   |   ├── badges.py                    # Badge definition CRUD, user badge holders, badge s
    tats
|   |   ├── categories.py                # Challenge categories listing and auto-seeding
|   |   ├── challenges.py                # Challenge CRUD, hints, writeups, and secure file d
    ownloads
|   |   ├── deps.py                      # Dependency injectors: get_db, auth tokens, role gu
    ards
|   |   ├── discussions.py               # Forum threads, comments, pin/lock/delete moderatio
    n
|   |   ├── events.py                    # CTF competitions, registrations, score freeze, exp
    ort, certs
|   |   ├── leaderboard.py               # Global, college, and user personal rank leaderboar
    ds
|   |   ├── live_challenges.py           # Live container sandboxes: launch, inspect, termina
    te, prune
|   |   ├── notifications.py             # User notifications list, unread counters, mark as
    read
|   |   ├── profile.py                   # Profile details, 365-day heatmap, solve history, s
    treaks
|   |   ├── submissions.py               # Flag submission processing, validation, personal s
    olve history
|   |   └── teams.py                     # Team creation, membership, captain transfer, disco
    very
|   ├── models/                          # 17 SQLAlchemy Relational Models (23 Tables Total)
|   |   ├── __init__.py                  # Model registry and declarative metadata binding
|   |   ├── activity.py                  # UserActivity model (timeline events feed)
|   |   ├── badge.py                     # Badge and UserBadge models (system achievements)
|   |   ├── base.py                      # Base declarative class and TimestampMixin
|   |   ├── category.py                  # Category model (Web, Crypto, Pwn, Reverse, etc.)
|   |   ├── challenge.py                 # Challenge, ChallengeFile, ChallengeHint, Challenge
    Writeup
|   |   ├── challenge_instance.py        # ChallengeInstance model (ephemeral Docker sandbox
    leases)
|   |   ├── discussion.py                # Discussion and DiscussionComment models
|   |   ├── event.py                     # Event and EventChallenge association models
|   |   ├── event_registration.py        # EventRegistration model (solo or team participatio
    n)
|   |   ├── hint_usage.py                # HintUsage model (unlock ledger and point penalty a
    udit)
|   |   ├── notification.py              # Notification model (in-app notifications and read
    flags)
|   |   ├── security_event.py            # SecurityEvent and SecurityEventReview audit models
|   |   ├── solve.py                     # Solve model (authoritative solved challenges ledge
    r)
|   |   ├── submission.py                # Submission model (every flag submission attempt)
|   |   ├── team.py                      # Team and TeamMember models
|   |   └── user.py                      # User model (accounts, credentials, roles, moderati
    on)
|   ├── schemas/                         # 18 Pydantic Request & Response Data Contracts
|   |   ├── __init__.py
|   |   ├── admin_user.py                # Admin user list, detail, and update schemas
|   |   ├── analytics.py                 # 16 Analytics domain response schemas
|   |   ├── auth.py                      # LoginRequest, TokenResponse
|   |   ├── badge.py                     # BadgeCreate, BadgeUpdate, BadgeResponse, UserBadge
    Response
|   |   ├── certificate.py               # CertificateResponse schema (HMAC-SHA256 signature
    verification)
|   |   ├── challenge.py                 # ChallengeCreate, ChallengeUpdate, ChallengeRespons
    e
|   |   ├── challenge_instance.py        # ChallengeInstanceResponse, AdminInstanceResponse
|   |   ├── discussion.py                # DiscussionCreate, CommentCreate, DiscussionRespons
    e
```

```
│   │   ├── event.py                    # EventCreate, EventUpdate, EventLeaderboard, Partic
    ipant
│   │   ├── hint.py                     # HintCreate, HintUpdate, HintResponse, HintUsageRes
    ponse
│   │   ├── leaderboard.py              # LeaderboardResponse, UserRankResponse
│   │   ├── notification.py             # NotificationResponse, NotificationListResponse
│   │   ├── profile.py                  # ProfileUpdate, Public/Private Profile, HeatmapDay,
     Streak
│   │   ├── security_event.py           # SecurityEventResponse, SecurityEventReviewRequest
│   │   ├── submission.py               # SubmissionCreate, SubmissionResponse, UserScoreRes
    ponse
│   │   ├── team.py                     # TeamCreate, TeamUpdate, TeamDetail, TeamMember
│   │   ├── user.py                     # UserRegisterRequest, UserResponse
│   │   └── writeup.py                  # WriteupCreate, WriteupUpdate, WriteupResponse
│   └── services/                       # 22 Domain Business Logic Services
│       ├── __init__.py
│       ├── activity_service.py         # User activity logging and timeline pagination
│       ├── analytics_service.py        # Deep metrics aggregation, trends, distributions
│       ├── anti_cheat_service.py       # Heuristic cheat detection, rapid solve alerts, IP
    monitoring
│       ├── auth_service.py             # User registration, bcrypt authentication, uniquene
    ss checks
│       ├── badge_service.py            # Badge rules evaluation, automated awards, progress
     tracking
│       ├── certificate_service.py      # Cryptographic certificate token generation & HMAC
    verification
│       ├── challenge_service.py        # Challenge CRUD, category seeding, file association
    s
│       ├── discussion_service.py       # Thread lifecycle, nested comment replies, moderati
    on actions
│       ├── docker_challenge_service.py # Docker container lifecycle, port allocation, expir
    ation sweeps
│       ├── event_export_service.py     # Event rankings, solves, and score exports (JSON an
    d CSV)
│       ├── event_service.py            # Event lifecycle, registrations, scoreboard freeze/
    unfreeze
│       ├── hint_service.py             # Tiered hint reveals, point penalty ledgering
│       ├── leaderboard_service.py      # Global and college leaderboard rankings with deter
    ministic tie-breaks
│       ├── notification_service.py     # In-app notifications creation, batch read, unread
    counts
│       ├── profile_service.py          # Aggregated student profiles, category solve distri
    butions
│       ├── scoring_service.py          # Authoritative points calculation, solves verificat
    ion, status
│       ├── storage_service.py          # Disk-based file storage with path sanitization
│       ├── streak_service.py           # Consecutive active days and historical longest str
    eaks
│       ├── submission_service.py       # Flag evaluation, rate limiting, duplicate solve pr
    otection
│       ├── team_service.py             # Team rosters, captaincy transfer, invite managemen
    t
│       ├── user_management_service.py  # Admin user discovery, role promotion, ban/mute tog
    gles
│       └── writeup_service.py          # Writeup authoring and post-solve access gating
├── challenges/                         # Challenge resource templates and Docker definition
    s
├── docs/                               # Production deployment and operational guides
│   ├── BACKUP_RESTORE.md               # Backup/restore policies and automation scripts
│   ├── DATABASE_SETUP.md               # Supabase PostgreSQL configuration & migration guid
    e
│   └── DEPLOYMENT_GUIDE.md             # Docker Compose production deployment instructions
├── frontend/                           # Static single-page application served directly by
    FastAPI
│   ├── css/style.css                   # Cybersecurity design system (dark-first, tokens, g
    rids)
```

```
│   ├── index.html                    # SPA application shell, navigation, modals
│   └── js/                           # Modular vanilla JavaScript client
│       ├── api.js                    # Centralized FastAPI client (JWT auth, error parsin
    g)
│       ├── app.js                    # Application bootstrap and client-side hash router
│       ├── state.js                  # Reactive state, modal utilities, and toast notific
    ations
│       └── views/                    # 9 Modular view controllers (landing, challenges, a
    dmin, etc.)
├── scripts/
│   └── create_admin.py               # Administrator account bootstrapping and promotion
    CLI
├── tests/                            # 22 Test Suites (233 Passing Automated Tests)
├── Dockerfile                        # Production multi-stage Docker container specificat
    ion
├── docker-compose.yml                # Unified multi-service deployment configuration
├── pytest.ini                        # Pytest configuration
└── requirements.txt                  # Python dependencies
```

2. Database Model Overview

The backend uses 23 application tables plus the Alembic migration version table. Every primary key uses standard RFC-4122 UUIDs.

Model Class	Database Table	Purpose	Key Fields	Relationships	Constraints	Frontend Relevance
`User`	`users`	User credentials, roles, profile, and moderation	`id`, `username`, `email`, `hashed_password`, `role`, `name`, `bio`, `college`, `avatar_url`, `is_active`, `is_verified`, `is_banned`, `is_muted`	`submissions`, `solves`, `team_memberships`, `activities`, `badges`, `notifications`	Unique `username`, Unique `email`. Indexes on `role`, `is_active`	Auth session, user badges, leaderboard rows, profile page, admin moderation
`Category`	`categories`	Challenge categories	`id`, `name`, `description`, `icon`, `is_active`	`challenges`	Unique `name`	Challenge filter pills, category distribution charts, landing page showcase
`Challenge`	`challenges`	CTF challenges catalog	`id`, `title`, `description`, `category_id`, `difficulty`, `points`, `flag`, `flag_hash`, `is_active`, `author`, `is_dynamic`, `is_live`, `docker_image`, `internal_port`, `protocol`	`category`, `hints`, `writeups`, `files`, `submissions`, `solves`, `discussions`	Foreign key `category_id`. Index on `is_active`, `difficulty`	Challenge browser, challenge modal, points tags, live Docker tags
`ChallengeFile`	`challenge_files`	Attachment files	`id`, `challenge_id`, `filename`, `file_size`, `storage_path`, `checksum`	`challenge`	Foreign key `challenge_id`	Challenge modal file attachment download button with file size
`ChallengeHint`	`challenge_hints`	Tiered hints with point penalties	`id`, `challenge_id`, `hint_text`, `cost`, `order`, `title`, `is_active`	`challenge`, `usages`	Foreign key `challenge_id`. Unique `(challenge_id, order)`	Hint accordion, unlock confirmation dialog, penalty warnings
`ChallengeWriteup`	`challenge_writeups`	Official author writeups	`id`, `challenge_id`, `author_id`, `title`, `content`, `is_published`	`challenge`, `author`	Foreign key `challenge_id`, `author_id`	Post-solve writeup tab (strictly hidden from students until solved)
`Submission`	`submissions`	Audit log of all attempts	`id`, `user_id`, `challenge_id`, `team_id`, `event_id`, `submitted_flag`, `is_correct`, `points_awarded`, `ip_address`	`user`, `challenge`, `team`, `event`	Foreign keys on all parents. Indexes on `(user_id, challenge_id)`	Real-time submission feedback, attempt counters, failure rates

Model Class	Database Table	Purpose	Key Fields	Relationships	Constraints	Frontend Relevance
Solve	solves	Unique authoritative solve ledger	id, user_id, challenge_id, team_id, event_id, points_awarded, created_at	user, challenge, team, event	Unique (user_id, challenge_id) per event/practice. Composite indexes	Solve checkmarks, green difficulty pills, leaderboard points, streak logs
HintUsage	hint_usage	Hint unlocks audit ledger	id, user_id, hint_id, team_id, cost_applied, created_at	user, hint, team	Unique (user_id, hint_id)	Disables unlock button once used; shows revealed hint text
Team	teams	Student teams	id, name, description, captain_id, college, invite_code, is_active, total_points	captain, members, submissions, solves, registrations	Unique name, Unique invite_code	Teams directory, team card, captain controls, member invite modal
TeamMember	team_members	Team membership mapping	id, team_id, user_id, role, joined_at	team, user	Unique (team_id, user_id). Team size max 4	Member roster list, leave button, remove member button
Event	events	Timed CTF competitions	id, title, slug, description, start_at, end_at, registration_start_at, registration_end_at, status, is_active, is_scoreboard_frozen, scoreboard_frozen_at, max_team_size	creator, challenges, registrations, submissions, solves	Unique slug. Index on start_at, end_at, status	Events hub, countdown tickers, event scoreboard, frozen indicator
EventChallenge	event_challenges	Event-challenge mapping	id, event_id, challenge_id, points_override, order	event, challenge	Unique (event_id, challenge_id)	Event challenge arena; event-specific points override display
EventRegistration	event_registrations	Event participants	id, event_id, user_id, team_id, registration_type, registered_at	event, user, team	Unique (event_id, user_id) and (event_id, team_id)	"Registered" badge, Register/Unregister button, certificates
UserActivity	user_activities	User action activity feed	id, user_id, activity_type, title, description, created_at	user	Foreign key user_id. Index on (user_id, created_at)	Student dashboard activity feed, operative profile activity list
Badge	badges	System achievements	id, slug, name, description, icon, category, points_requirement, solves_requirement, is_active	user_badges	Unique slug	Badges catalog, locked vs unlocked badge cards
UserBadge	user_badges	Awarded badges ledger	id, user_id, badge_id, earned_at	user, badge	Unique (user_id, badge_id)	Earned badge gold borders, award date tooltips, profile showcase
Discussion	discussions	Challenge topics	id, challenge_id, user_id, title, content, is_pinned, is_locked, has_spoilers, is_deleted	challenge, user, comments	Foreign key challenge_id, user_id. Index on created_at	Forum thread list, pinned badges, locked thread notices
DiscussionComment	discussion_comments	Discussion replies	id, discussion_id, user_id, parent_id, content, is_deleted	discussion, user, replies	Foreign key discussion_id, parent_id	Threaded reply tree, delete button, author handles
Notification	notifications	In-app user alerts	id, user_id, type, title, message, data, is_read, created_at	user	Foreign key user_id. Index on (user_id, is_read)	Top nav bell unread counter, notification slide-over drawer
SecurityEvent	security_events	Anti-cheat alerts	id, user_id, challenge_id, event_type, severity, status, details, ip_address, created_at	user, challenge, reviews	Foreign keys. Index on severity, status, event_type	Admin anti-cheat incident table, severity badges (critical/high)

Model Class	Database Table	Purpose	Key Fields	Relationships	Constraints	Frontend Relevance
`SecurityEventReview`	`security_event_reviews`	Incident review log	`id`, `security_event_id`, `reviewer_id`, `previous_status`, `new_status`, `note`, `created_at`	`security_event`, `reviewer`	Foreign key `security_event_id`, `reviewer_id`	Review audit trail modal, moderator resolution notes
`ChallengeInstance`	`challenge_instances`	Live container leases	`id`, `challenge_id`, `user_id`, `team_id`, `event_id`, `container_id`, `container_name`, `status`, `internal_port`, `external_port`, `external_host`, `url`, `started_at`, `expires_at`	`challenge`, `user`, `team`, `event`	Unique `external_port`. Index on `status`, `expires_at`	Launch/Terminate sandbox buttons, ticking countdown timer, URL link

3. Authentication & Authorization

3.1 Authentication Architecture

- Password Security: Bcrypt with unique salt generation (`bcrypt.gensalt()`).
- Token Mechanism: Standard JWT (JSON Web Tokens) encoded via `python-jose` with `HS256`. Token payload contains:
- `sub`: User ID (UUID string)
- `email`: User email address
- `role`: Role string (`student`, `moderator`, `admin`)
- `exp`: Expiration epoch (default 1440 minutes / 24 hours)
- Token Transmission: `Authorization: Bearer <JWT>` header in HTTP requests.
- Client Storage: `localStorage.getItem('owasp_jwt')`.
- Session Restoration: Handled at application bootstrap by calling `GET /api/auth/me`. If the token is invalid or expired (HTTP 401), the frontend cleans storage and transitions to unauthenticated state.

3.2 User Roles & Hierarchy

```
ADMIN
  └── Can access all features, platform analytics, user moderation (ban/mute/role),
      and system cleanup.
MODERATOR
  └── Can manage challenges, writeups, hints, events, moderate discussions,
      and review security incidents.
STUDENT
  └── Standard participant. Can solve challenges, join teams, register for events,
      earn badges, post in discussions, and view leaderboards.
```

- Student Protection Rule: Public registration (`POST /api/auth/register`) strictly sets `role: "student"`. Roles cannot be elevated through public APIs.
- Moderation Fields:
- `is_banned`: Banned users are denied JWT authentication and access.
- `is_muted`: Muted users cannot create discussions or post comments (`HTTP 403`).

4. API Endpoint Reference

4.1 Authentication (`/api/auth`)

Method	Endpoint	Authentication	Role	Purpose
POST	/api/auth/register	None	Public	Register new student account (email must be valid domain)
POST	/api/auth/login	None	Public	Authenticate credentials (email, password) and receive JWT
GET	/api/auth/me	Bearer JWT	Any active	Fetch currently authenticated user profile and permissions
GET	/api/auth/me/score	Bearer JWT	Any active	Fetch caller's authoritative total score and solve count
GET	/api/auth/test/student	Bearer JWT	Student+	RBAC validation endpoint
GET	/api/auth/test/moderator	Bearer JWT	Moderator+	RBAC validation endpoint
GET	/api/auth/test/admin	Bearer JWT	Admin only	RBAC validation endpoint

4.2 Challenges & Categories (/api/categories, /api/challenges)

Method	Endpoint	Authentication	Role	Purpose
GET	/api/categories	None	Public	List all challenge categories (auto-seeds defaults if empty)
GET	/api/challenges	Optional	Public/Auth	List practice challenges (strips flags from non-admin responses)
GET	/api/challenges/{id}	Optional	Public/Auth	Retrieve single challenge prompt, difficulty, author, points
GET	/api/challenges/{id}/status	Bearer JWT	Student+	Get challenge solve state (solved, attempted, unsolved)
POST	/api/challenges	Bearer JWT	Moderator+	Create a new challenge
PATCH	/api/challenges/{id}	Bearer JWT	Moderator+	Update challenge details, points, or flag
DELETE	/api/challenges/{id}	Bearer JWT	Moderator+	Soft deactivate or hard delete challenge (?hard=true)
POST	/api/challenges/{id}/files	Bearer JWT	Moderator+	Upload challenge attachment file
GET	/api/challenges/{id}/files/{file_id}/download	Optional	Public/Auth	Download challenge attachment file safely

4.3 Submissions & Scoring (/api/submissions)

Method	Endpoint	Authentication	Role	Purpose
POST	/api/submissions	Bearer JWT	Student+	Submit flag candidate for evaluation; receives points if correct
GET	/api/submissions/me	Bearer JWT	Student+	Get caller's personal submission attempt history (max 100)

4.4 Hints & Writeups (/api/challenges/{id}/hints, /api/challenges/{id}/writeups)

Method	Endpoint	Authentication	Role	Purpose
GET	/api/challenges/{id}/hints	Optional	Public/Auth	List hints (reveals content only if unlocked by student or caller is admin)
GET	/api/challenges/{id}/hints/{hint_id}	Optional	Public/Auth	Retrieve single hint
POST	/api/challenges/{id}/hints/{hint_id}/use	Bearer JWT	Student+	Unlock hint; deducts cost from student score and reveals text
POST	/api/challenges/{id}/hints	Bearer JWT	Moderator+	Add new tiered hint with point penalty cost
PUT	/api/challenges/{id}/hints/{hint_id}	Bearer JWT	Moderator+	Modify hint title, text, cost, or order
DELETE	/api/challenges/{id}/hints/{hint_id}	Bearer JWT	Moderator+	Delete a hint
GET	/api/challenges/{id}/writeup	Optional	Public/Auth	Get writeup (locked with HTTP 403 for students until solved)
GET	/api/challenges/{id}/writeups	Optional	Public/Auth	List all published writeups for challenge
POST	/api/challenges/{id}/writeups	Bearer JWT	Moderator+	Publish official author solution writeup
PUT	/api/challenges/{id}/writeups/{writeup_id}	Bearer JWT	Moderator+	Update writeup title or markdown content
DELETE	/api/challenges/{id}/writeups/{writeup_id}	Bearer JWT	Moderator+	Delete writeup

4.5 Leaderboards (`/api/leaderboard`)

Method	Endpoint	Authentication	Role	Purpose
GET	/api/leaderboard/global	None	Public	Paginated global leaderboard with deterministic ranking
GET	/api/leaderboard/college	None	Public	Aggregated college/university standings
GET	/api/leaderboard/me	Bearer JWT	Student+	Caller's current global rank and nearby competitors

4.6 Teams (`/api/teams`)

Method	Endpoint	Authentication	Role	Purpose
POST	/api/teams	Bearer JWT	Student+	Create a new squad (caller becomes Captain / Owner)
GET	/api/teams	None	Public	List and search squads (by name or college)
GET	/api/teams/{team_id}	None	Public	Get team card, points, captain name, and member roster
PUT	/api/teams/{team_id}	Bearer JWT	Captain/Admin	Update team name or description
DELETE	/api/teams/{team_id}	Bearer JWT	Captain/Admin	Soft-deactivate team
GET	/api/teams/{team_id}/members	None	Public	List active roster members
POST	/api/teams/{team_id}/members	Bearer JWT	Captain/Admin	Add member by username or user_id (max 4 members)
DELETE	/api/teams/{team_id}/members/{user_id}	Bearer JWT	Captain/Admin	Kick member from team
DELETE	/api/teams/{team_id}/leave	Bearer JWT	Student+	Non-captain member leaves team
POST	/api/teams/{team_id}/transfer-owner	Bearer JWT	Captain only	Transfer captaincy to another member

4.7 Events & Tournaments (`/api/events`)

Method	Endpoint	Authentication	Role	Purpose
GET	/api/events	Optional	Public/Auth	List upcoming, live, and concluded CTF tournaments
GET	/api/events/{id_or_slug}	Optional	Public/Auth	Get event metadata, schedule, and window
POST	/api/events	Bearer JWT	Moderator+	Create a new CTF event
PUT	/api/events/{id}	Bearer JWT	Moderator+	Update event schedule, status, or description
DELETE	/api/events/{id}	Bearer JWT	Moderator+	Delete an event
POST	/api/events/{id}/register	Bearer JWT	Student+	Register for event (as solo individual or team captain)
POST	/api/events/{id}/unregister	Bearer JWT	Student+	Withdraw registration before event closes
GET	/api/events/{id}/registration	Bearer JWT	Student+	Check registration status for authenticated user/team
GET	/api/events/{id}/participants	None	Public	List registered event participants
GET	/api/events/{id}/challenges	Optional	Public/Auth	List event challenges (locked until event is live)
POST	/api/events/{id}/challenges	Bearer JWT	Moderator+	Assign challenge to event with custom points
DELETE	/api/events/{id}/challenges/{challenge_id}	Bearer JWT	Moderator+	Remove challenge from event
POST	/api/events/{id}/submissions	Bearer JWT	Student+	Submit flag for an event challenge
GET	/api/events/{id}/leaderboard	Optional	Public/Auth	Isolated event scoreboard (respects scoreboard freeze)
POST	/api/events/{id}/freeze-scoreboard	Bearer JWT	Moderator+	Freeze public scoreboard updates
POST	/api/events/{id}/unfreeze-scoreboard	Bearer JWT	Moderator+	Resume live public scoreboard updates
GET	/api/events/{id}/export	Bearer JWT	Moderator+	Export event results as JSON or CSV (`?format=csv`)
GET	/api/events/{id}/certificate	Bearer JWT	Student+	Claim verifiable digital achievement certificate

Method	Endpoint	Authentication	Role	Purpose
GET	/api/events/certificates/verify/{token}	None	Public	Public cryptographic certificate token verification

4.8 Profile & Activity (/api/profile)

Method	Endpoint	Authentication	Role	Purpose
GET	/api/profile/me	Bearer JWT	Student+	Private profile (score, rank, bio, college, email)
PUT	/api/profile/me	Bearer JWT	Student+	Update display name, college, bio, avatar URL
GET	/api/profile/me/solved	Bearer JWT	Student+	Detailed list of caller's solved challenges
GET	/api/profile/me/activity	Bearer JWT	Student+	Paginated activity timeline feed
GET	/api/profile/me/activity/heatmap	Bearer JWT	Student+	Daily solve counts for 365-day GitHub heatmap
GET	/api/profile/me/streak	Bearer JWT	Student+	Current active day streak and longest historical streak
GET	/api/profile/me/category-stats	Bearer JWT	Student+	Solves and points broken down by category
GET	/api/profile/me/badges	Bearer JWT	Student+	Unlocked achievement badges with award timestamps
GET	/api/profile/me/badge-progress	Bearer JWT	Student+	Progress toward unlocking remaining badges
GET	/api/profile/{user_id}	Optional	Public/Auth	Public profile with statistics for any operative
GET	/api/profile/{user_id}/solved	Optional	Public/Auth	Publicly verified solves for target user
GET	/api/profile/{user_id}/activity	Optional	Public/Auth	Public activity stream for target user
GET	/api/profile/{user_id}/activity/heatmap	Optional	Public/Auth	365-day heatmap for target user
GET	/api/profile/{user_id}/streak	Optional	Public/Auth	Streak statistics for target user
GET	/api/profile/{user_id}/category-stats	Optional	Public/Auth	Category statistics for target user
GET	/api/profile/{user_id}/badges	Optional	Public/Auth	Unlocked badges for target user

4.9 Badges Catalog (/api/badges)

Method	Endpoint	Authentication	Role	Purpose
GET	/api/badges	None	Public	List all active achievement badges in system
GET	/api/badges/{badge_id}	None	Public	Get single badge details by UUID or slug
GET	/api/badges/{badge_id}/stats	None	Public	Get award count and statistics for badge
POST	/api/badges	Bearer JWT	Admin only	Define a new platform badge
PUT	/api/badges/{badge_id}	Bearer JWT	Admin only	Update badge criteria or presentation
DELETE	/api/badges/{badge_id}	Bearer JWT	Admin only	Deactivate or delete badge
GET	/api/badges/{badge_id}/users	Bearer JWT	Admin only	List all users who have earned this badge

4.10 Discussions & Forum (/api/challenges/{id}/discussions, /api/discussions, /api/comments)

Method	Endpoint	Authentication	Role	Purpose
GET	/api/challenges/{id}/discussions	Optional	Public/Auth	List discussion threads for a challenge (pinned first)
POST	/api/challenges/{id}/discussions	Bearer JWT	Student+	Create topic (rejected with 403 if user is muted)
GET	/api/discussions/{discussion_id}	Optional	Public/Auth	Get thread details and comments
PUT	/api/discussions/{discussion_id}	Bearer JWT	Author/Mod	Edit discussion title or content
DELETE	/api/discussions/{discussion_id}	Bearer JWT	Author/Mod	Soft-delete discussion

Method	Endpoint	Authentication	Role	Purpose
POST	/api/discussions/{id}/pin	Bearer JWT	Moderator+	Pin/unpin discussion to top
POST	/api/discussions/{id}/lock	Bearer JWT	Moderator+	Lock/unlock thread to prevent new comments
POST	/api/discussions/{id}/comments	Bearer JWT	Student+	Post comment or nested reply
PUT	/api/comments/{comment_id}	Bearer JWT	Author/Mod	Edit comment text
DELETE	/api/comments/{comment_id}	Bearer JWT	Author/Mod	Soft-delete comment

4.11 Notifications (/api/notifications)

Method	Endpoint	Authentication	Role	Purpose
GET	/api/notifications	Bearer JWT	Student+	Paginated notifications (?unread_only=true)
GET	/api/notifications/unread-count	Bearer JWT	Student+	Get number of unread alerts for bell badge
POST	/api/notifications/{id}/read	Bearer JWT	Student+	Mark single notification as read
POST	/api/notifications/read-all	Bearer JWT	Student+	Mark all notifications as read
DELETE	/api/notifications/{id}	Bearer JWT	Student+	Delete notification

4.12 Live Docker Challenge Sandboxes (/api/challenges/{id}/instance, /api/admin/challenge-instances)

Method	Endpoint	Authentication	Role	Purpose
POST	/api/challenges/{id}/instance	Bearer JWT	Student+	Launch or retrieve running live container instance
GET	/api/challenges/{id}/instance	Bearer JWT	Student+	Check status, host, port, URL, and expiration timer
DELETE	/api/challenges/{id}/instance	Bearer JWT	Student+	Manually terminate student's active sandbox
GET	/api/admin/challenge-instances	Bearer JWT	Moderator+	List all active container instances across all students
POST	/api/admin/challenge-instances/{id}/stop	Bearer JWT	Moderator+	Force terminate a container instance
POST	/api/admin/challenge-instances/cleanup	Bearer JWT	Moderator+	Prune expired container instances and reclaim ports

4.13 Security & Anti-Cheat (/api/admin/security)

Method	Endpoint	Authentication	Role	Purpose
GET	/api/admin/security/events	Bearer JWT	Moderator+	Filterable log of security incidents & anomalies
GET	/api/admin/security/events/{id}	Bearer JWT	Moderator+	Get security incident details and audit trail
POST	/api/admin/security/events/{id}/review	Bearer JWT	Moderator+	Triage incident (actioned, dismissed, reviewed)
GET	/api/admin/security/users/{user_id}	Bearer JWT	Moderator+	Computed security risk score and anomaly breakdown

4.14 Admin User Moderation (/api/admin/users)

Method	Endpoint	Authentication	Role	Purpose
GET	/api/admin/users	Bearer JWT	Moderator+	Search and list users with score, solves, and status
GET	/api/admin/users/{user_id}	Bearer JWT	Moderator+	Full user audit details and recent submissions
PATCH	/api/admin/users/{user_id}	Bearer JWT	Admin only	Update role (admin/moderator/student), ban, or mute

4.15 Admin Analytics (/api/admin/analytics)

Method	Endpoint	Authentication	Role	Purpose
GET	/api/admin/analytics/overview	Bearer JWT	Moderator+	High-level platform health and volume KPIs
GET	/api/admin/analytics/users	Bearer JWT	Moderator+	User growth, college distribution, and solve breakdown
GET	/api/admin/analytics/activity	Bearer JWT	Moderator+	Daily activity frequencies and timeline trends
GET	/api/admin/analytics/challenges	Bearer JWT	Moderator+	Solve rates and attempt counts for each challenge
GET	/api/admin/analytics/challenges/difficulty	Bearer JWT	Moderator+	Aggregate solve rates grouped by difficulty
GET	/api/admin/analytics/challenges/insights	Bearer JWT	Moderator+	Hardest, most solved, and most attempted challenges
GET	/api/admin/analytics/categories	Bearer JWT	Moderator+	Category distributions of points and solvers
GET	/api/admin/analytics/submissions	Bearer JWT	Moderator+	Submission volumes, accuracy rate, failure spikes
GET	/api/admin/analytics/events	Bearer JWT	Moderator+	Aggregate event participation metrics
GET	/api/admin/analytics/events/{id}	Bearer JWT	Moderator+	Deep-dive analytics for specific event
GET	/api/admin/analytics/teams	Bearer JWT	Moderator+	Team sizes, rosters, and collegiate affiliations
GET	/api/admin/analytics/badges	Bearer JWT	Moderator+	Badge popularity and unlock rates
GET	/api/admin/analytics/discussions	Bearer JWT	Moderator+	Community activity and moderation volumes
GET	/api/admin/analytics/leaderboard	Bearer JWT	Moderator+	Score distribution histograms, mean/median points
GET	/api/admin/analytics/dashboard	Bearer JWT	Moderator+	Unified dashboard metric bundle

4.16 System Health Probes

Method	Endpoint	Authentication	Role	Purpose
GET	/health	None	Public	Liveness probe: returns service status and environment
GET	/health/ready	None	Public	Readiness probe: executes `SELECT 1` against PostgreSQL

---

5. Challenge System

1. Browsing & Discovery:

- Filter by Category (Web Exploitation, Cryptography, Reverse Engineering, Binary Exploitation, Digital Forensics, OSINT, Steganography, Miscellaneous).
- Filter by Difficulty: `Easy`, `Medium`, `Hard`, `Insane`.
- Filter by Status: Solved vs. Unsolved.
- Search by keyword in title or description.

2. Challenge Security:

- `flag` and `flag_hash` are never returned to students; the backend strips them from responses.
- Inactive challenges return HTTP 404 for students, visible only to admins/moderators.

3. Practice vs. Event Scope:

- Practice challenges can be solved at any time.
- Event challenges are locked until the event start timestamp and require verified registration.

4. Live Container Lifecycle:

- Live challenges have `is_live = true`, `docker_image`, and port mappings.
- Starting an instance calls `POST /api/challenges/{id}/instance`, allocating an isolated Docker container with an external port and expiration timer.
- Students receive connection details: `http://<host>:<port>` or `nc <host> <port>`.
- When the TTL expires, the backend prunes the container. Students can also terminate it manually.

---

6. Scoring System

[!IMPORTANT] Authoritative Rule: The frontend must NEVER calculate scores or points locally. All scoring calculations are performed strictly by `ScoringService` and `SubmissionService` in the backend.

- Base Points: Challenges define a base point reward (e.g. 100, 250, 500 PTS).
- Correct Submission:
- Validates submitted flag using constant-time hash comparison (`secrets.compare_digest`).
- Checks the unique `solves` ledger.
- If valid and un-solved: records solve, calculates hint deductions, awards points, and updates user score.
- Duplicate Solve Protection:
- Submitting a flag for an already solved challenge returns `is_correct: true`, `points_awarded: 0`, message: "Challenge already solved". Duplicate points are never awarded.
- Hint Deductions:
- Each hint defines a point penalty cost.
- Unlocking a hint creates a permanent `hint_usage` audit record.
- The penalty is deducted from the challenge points awarded upon solving.
- Tie-Breaking Determinism:
- When two competitors have identical scores, rank is determined strictly by who reached that score first (`earliest last_solve_time`).
- Scoreboard Freezing:
- During live events, administrators can freeze the scoreboard.
- Public leaderboard endpoints freeze their rankings at the freeze timestamp.
- Solves continue recording silently in the backend database.

7. Leaderboards

1. **Global Individual Leaderboard (`GET /api/leaderboard/global`)**:
- Ranks all students across the platform.
- Supports category filtering (`?category=Web Exploitation`).
- Supports time filtering: `all_time`, `today`, `this_week`, `this_month`.
- Returns paginated entries: `rank`, `username`, `college`, `total_points`, `solves_count`, `last_solve_time`.
2. **College Standings (`GET /api/leaderboard/college`)**:
- Aggregates points of all students representing the same college.
- Displays university rank, total points, and active competitor count.
3. **User Rank Card (`GET /api/leaderboard/me`)**:
- Returns caller's exact global rank, points, and immediate competitors above and below.
4. **Event Scoreboard (`GET /api/events/{id}/leaderboard`)**:
- Scoped entirely to a specific competition.
- Shows "SCOREBOARD FROZEN" banner if freeze is active.

8. Teams

- Team Creation: Any student can create a team via `POST /api/teams`. The creator automatically becomes the Captain.
- Max Team Size: Enforced strictly at 4 members.
- Invite Codes: Teams have a secret `invite_code` used by captains to recruit members.
- Member Roles: Captain (`owner`) vs. Member (`member`).

- Captain Permissions:
- Add members (`POST /api/teams/{id}/members`).
- Kick members (`DELETE /api/teams/{id}/members/{user_id}`).
- Transfer ownership (`POST /api/teams/{id}/transfer-owner`).
- Disband/deactivate team (`DELETE /api/teams/{id}`).
- Member Permissions:
- Normal members can leave the team via `DELETE /api/teams/{id}/leave`. Captains cannot leave without transferring ownership first.

---

9. Events & Tournaments

- Event States:
- `UPCOMING`: Countdown active, registrations open.
- `LIVE`: Competition running, challenge arena accessible to registrants.
- `ENDED`: Challenges closed, final leaderboard published, writeups unlocked.
- Registration:
- Solo individual or Team registration (captains register their squad).
- Enforces registration window (`registration_start_at` to `registration_end_at`).
- Results Export:
- Admins can export final event standings in JSON or CSV format (`GET /api/events/{id}/export?format=csv`).
- Verifiable Digital Certificates:
- Participants can claim a digital certificate (`GET /api/events/{id}/certificate`).
- Generates an HMAC-SHA256 signed verification token.
- Any employer or recruiter can verify the token publicly at `GET /api/events/certificates/verify/{token}`.

---

10. Profiles & Activity

- **Private Profile (`GET /api/profile/me`)**:
- Operative identity: Avatar, display name, handle, college, bio.
- Key stats: Total score, global rank, college rank, solve count.
- Streak tracker: Current streak (consecutive days) and longest historical streak.
- **Public Profile (`GET /api/profile/{user_id}`)**:
- Accessible to other operatives; hides email and private fields.
- 365-Day Activity Heatmap:
- `GET /api/profile/me/activity/heatmap` returns daily solve counts.
- Rendered as a 52-week GitHub-style heatmap grid with 5 intensity levels (`0`, `level-1`, `level-2`, `level-3`, `level-4`).
- Category Radar/Bar Stats:
- `GET /api/profile/me/category-stats` returns solve counts and points earned in each category.

---

11. Badges & Achievements

1. System Badges Seeded in Database:
- `first-blood`: First solver of any challenge.
- `web-master`: Solved 5+ Web Exploitation challenges.
- `crypto-cracked`: Solved 5+ Cryptography challenges.

- `pwn-star`: Solved 3+ Binary Exploitation challenges.
- `forensics-detective`: Solved 5+ Digital Forensics challenges.
- `reverse-engineer`: Solved 3+ Reverse Engineering challenges.
- `streak-master`: Maintained a 7-day active CTF streak.
- `top-ten`: Placed in the top 10 on the global leaderboard.

2. Automatic Awarding:

- Evaluated automatically on every successful flag submission.
- Generates an in-app notification when a new badge is unlocked.

3. Progress Tracking:

- `GET /api/profile/me/badge-progress` returns the student's completion percentage (e.g. `2/5 challenges solved, 40%`) toward locked badges.

---

12. Discussions & Forum

- Contextual Challenge Threads:
- Discussion topics are linked to specific challenges.
- Lists threads sorted by `newest`, `oldest`, or `pinned`.
- Comment Trees:
- Supports top-level comments and nested replies (`parent_id`).
- Anti-Spoiler Protection:
- Topics can be flagged with `has_spoilers: true` to collapse content by default.
- Moderation Actions:
- Moderators/Admins can Pin, Lock, or Delete topics.
- Soft-deletion preserves discussion structure while masking content (`[Comment deleted]`).
- Mute Enforcement:
- Users with `is_muted: true` receive HTTP 403 when attempting to create topics or comment.

---

13. Notifications

- Types: `SOLVE`, `FIRST_BLOOD`, `BADGE_UNLOCKED`, `TEAM_INVITE`, `EVENT_START`, `SECURITY_ALERT`.
- Navigation Placement:
- Top header bell icon with red numeric unread badge (`GET /api/notifications/unread-count`).
- Slide-over drawer with list of recent notifications (`GET /api/notifications`).
- One-click "Mark All Read" (`POST /api/notifications/read-all`).

---

14. Admin Analytics

The admin analytics suite is mounted at `/api/admin/analytics` and accessible strictly to `admin` and `moderator` roles:

1. **Overview (`/overview`)**: Total users, total challenges, total solves, active streak averages.
2. **User Metrics (`/users`)**: Registration trends, collegiate representation breakdown.
3. **Activity Trends (`/activity`)**: Daily solve volume over 7d, 30d, or all-time periods.
4. **Challenge Performance (`/challenges`, `/challenges/difficulty`, `/challenges/insights`)**: Hardest challenges by failure rate, fastest solves, and solve distributions.
5. **Submission Accuracy (`/submissions`)**: Global correct vs. incorrect flag submission ratios.
6. **Collegiate & Team Stats (`/teams`, `/categories`)**: Category coverage and team size distribution.
7. **Score Distribution (`/leaderboard`)**: Competitor point spread histograms (mean, median, standard deviation).

15. Security & Anti-Cheat

The platform includes automated heuristic cheat detection:

- Rate Limiting: Throttles brute-force submission attempts per user and IP address.
- Anomaly Detection:
- Duplicate flag submissions across multiple accounts.
- Impossible solve velocities (solving complex challenges in < 15 seconds).
- Multiple user accounts submitting from identical IP addresses.
- Incident Severity Levels: `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`.
- Moderator Triage Interface:
- Review incidents at `/api/admin/security/events`.
- Mark status: `OPEN`, `REVIEWED`, `DISMISSED`, `ACTIONED`.
- Add reviewer notes to the immutable audit trail.
- Note: Anti-cheat events flag suspicious behavior for administrative review; they do not silently delete valid solves without moderator intervention.

16. Live Challenge System

- Concept: Ephemeral Docker containers for web exploitation, pwn, or service challenges where competitors need live networked targets.
- Isolation: Per-user or per-team isolated instances with unique ports (`30000-35000`).
- Lifecycle:
- Student clicks "Launch Instance" → `POST /api/challenges/{id}/instance`.
- Container status transitions: `STARTING` → `RUNNING`.
- Backend provides access URL (`http://challenges.domain:31245`).
- Expiration timer (default 30–60 minutes TTL).
- Student or admin can terminate via `DELETE /api/challenges/{id}/instance`.
- Background task sweeps expired instances and releases host ports.

17. Security & Deployment

- Input Validation: Strict Pydantic schemas validating all incoming JSON bodies, UUID formats, string lengths, and email RFC specifications.
- CORS: Configurable allowed origins (`settings.CORS_ORIGINS`).
- Security Headers:
- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `Permissions-Policy: accelerometer=(), camera=(), geolocation=(), ...`
- Content Security Policy (CSP)
- `Strict-Transport-Security` (HSTS) when enabled
- IDOR Protection: All resource updates (teams, profiles, discussions, notifications) verify object ownership against the authenticated JWT identity before applying mutations.
- Safe File Delivery: File attachment downloads resolve through `StorageService` using sanitized UUID storage paths, preventing directory traversal attacks.

18. Required Frontend Pages

18.1 Public Pages

1. **Landing Page (`#landing`)**:
- Hero banner with action CTAs.
- Platform feature overview (Jeopardy arena, live Docker containers, digital certificates).
- Category showcase with solve counters.
- Upcoming events preview.
- Top 5 leaderboard podium preview.
- Public certificate verification widget.
2. **Public Leaderboard (`#leaderboard`)**:
- Global individual rankings and College rankings tabs.
- Top 3 Gold/Silver/Bronze podium cards.
- Full rankings table with pagination.
3. **Public Operative Profile (`#profile/{user_id}`)**:
- Operative bio, college affiliation, total points, global rank, and earned badges.
- 365-day activity heatmap and solved challenges list.

18.2 Student Pages (Authenticated)

4. **Student Dashboard (`#dashboard`)**:
- KPI metrics ribbon: Total score, global rank, solve count, active streak.
- Quick action shortcuts: "Browse Challenges", "Upcoming Events", "Team Roster".
- Real-time solve activity feed.
- Earned badges showcase.
5. **Challenge Arena (`#challenges`)**:
- Search bar and category filter pills.
- Difficulty dropdown (Easy, Medium, Hard, Insane) and status toggles (All, Solved, Unsolved).
- Challenge cards grid showing points, tags, solve status, and live container indicator.
- Challenge Detail Modal:
- Description with markdown rendering.
- Attachment download button.
- Live Docker container launcher and countdown timer.
- Hints accordion with point penalty warnings and unlock confirmations.
- Flag submission input with instant feedback.
- Post-solve author writeup tab.
- Direct jump to challenge discussion thread.
6. **Events & Tournament Hub (`#events`)**:
- Calendar of Upcoming, Live, and Concluded CTF events.
- Event cards with ticking countdown timers (`Starts in 02d 14h` / `Ends in 01h 30m`).
- Registration modal (solo or team captain).
- Event-assigned challenges arena.
- Isolated event scoreboard (with "SCOREBOARD FROZEN" banner when active).
- Claim digital certificate modal.
7. **Teams & Squads Hub (`#teams`)**:
- Squad directory and college search.
- "My Team" card with captain crown badge, roster list (up to 4 members), and secret invite code.
- Create Team modal and Join via Code modal.

- Captain management actions: Kick member, transfer captaincy, disband team.

8. **My Profile & Heatmap (`#profile`)**:

- Display name, college, avatar, and editable bio modal.
- 365-day GitHub-style solve intensity heatmap grid.
- Category solve distribution radar/bars.
- Badges & Achievements catalog with locked/unlocked indicators.
- Session logout button.

9. **Challenge Discussions & Debrief (`#discussions`)**:

- Challenge forum threads list.
- Thread view with author pills, timestamps, and nested reply trees.
- Create discussion modal with spoiler tag toggle.

10. **Notifications Center (`#notifications` or Drawer)**:

- Slide-over drawer accessible from the top navigation bell.
- Unread alert counter badge.
- Filter by all vs. unread notifications.
- One-click "Mark All Read" action.

18.3 Moderator & Admin Pages

11. **Admin Control Center (`#admin`)**:

- Restricted to `admin` and `moderator` roles.
- Tab 1: Analytics Dashboard: Platform health KPIs, category distribution, challenge insights.
- Tab 2: Challenge Authoring: Create, edit, and deactivate challenges; upload file attachments; manage hints and writeups.
- Tab 3: Event Management: Schedule competitions, set start/end windows, freeze/unfreeze scoreboard, export results (JSON/CSV).
- Tab 4: Anti-Cheat & Security: Incident monitor, severity filters, reviewer action modal.
- Tab 5: Live Docker Sandboxes: Active container monitor, container resource inspection, force kill switch, expired instance pruner.
- Tab 6: User Management (Admin Only): Operative search, college filters, role promotion (`admin`/`moderator`/`student`), ban/unban toggle, mute/unmute toggle.

---

19. Frontend Page → Backend API Mapping

Frontend Page / Modal	Backend API Endpoints Consumed	Main Data & Visual Elements
Landing Page	`GET /api/events` `GET /api/leaderboard/global?limit=5` `GET /api/categories` `GET /api/events/certificates/verify/{token}`	Top 5 players podium, upcoming competition teasers, category counters, certificate verification badge
Student Dashboard	`GET /api/profile/me` `GET /api/profile/me/streak` `GET /api/profile/me/activity` `GET /api/profile/me/badges` `GET /api/notifications/unread-count`	KPI metric cards (score, rank, streak), recent solve stream, earned badge ribbon, unread alert indicator
Challenge Browser	`GET /api/categories` `GET /api/challenges` `GET /api/challenges/{id}/status`	Category pills, difficulty badges, solve checkmarks, search filtering, points tags, live container indicators
Challenge Detail Modal	`GET /api/challenges/{id}` `GET /api/challenges/{id}/hints` `POST /api/challenges/{id}/hints/{hid}/use` `POST /api/submissions` `GET /api/challenges/{id}/writeup` `POST /api/challenges/{id}/instance` `DELETE /api/challenges/{id}/instance`	Markdown challenge prompt, file download button, hint unlock confirmation, flag submission form, sandbox controller, solution writeup

Frontend Page / Modal	Backend API Endpoints Consumed	Main Data & Visual Elements
Events Hub	`GET /api/events` `GET /api/events/{id}` `POST /api/events/{id}/register` `GET /api/events/{id}/challenges` `GET /api/events/{id}/leaderboard` `GET /api/events/{id}/certificate`	Countdown timers, registration toggles, event challenges list, event scoreboard, certificate claim modal
Leaderboards	`GET /api/leaderboard/global` `GET /api/leaderboard/college` `GET /api/leaderboard/me`	Top 3 Gold/Silver/Bronze podium, paginated ranking table, university standings, active user highlight row
Teams Hub	`GET /api/teams` `POST /api/teams` `GET /api/teams/{id}` `POST /api/teams/{id}/members` `DELETE /api/teams/{id}/leave` `POST /api/teams/{id}/transfer-owner`	Teams directory, captain crown, roster list (max 4), secret invite code, kick member button, transfer captaincy
Operative Profile	`GET /api/profile/me` `PUT /api/profile/me` `GET /api/profile/me/activity/heatmap` `GET /api/profile/me/category-stats` `GET /api/profile/me/badges` `GET /api/profile/me/badge-progress`	Operative bio, 365-day heatmap grid, category solve distribution, locked/unlocked badge catalog, logout action
Discussions Forum	`GET /api/challenges/{id}/discussions` `POST /api/challenges/{id}/discussions` `GET /api/discussions/{id}` `POST /api/discussions/{id}/comments` `POST /api/discussions/{id}/pin`	Thread list, pinned topic indicators, comment trees, spoiler reveal toggles, moderator lock/pin buttons
Notification Drawer	`GET /api/notifications` `GET /api/notifications/unread-count` `POST /api/notifications/{id}/read` `POST /api/notifications/read-all`	Unread counter badge, notification card stream, timestamp formatters, mark-all-read button
Admin Control Center	`GET /api/admin/analytics/dashboard` `POST /api/challenges` `POST /api/events` `POST /api/events/{id}/freeze-scoreboard` `GET /api/events/{id}/export` `GET /api/admin/security/events` `POST /api/admin/security/events/{id}/review` `GET /api/admin/challenge-instances` `GET /api/admin/users` `PATCH /api/admin/users/{id}`	Analytics KPI charts, challenge creation forms, event scheduler, anti-cheat incident reviews, live container monitor, user moderation table

20. Reusable Frontend Components

1. Header & Navigation:

- `BrandBadge`: Cybersecurity brand mark (`OWASP // CYBERLAB`).
- `NavPills`: Desktop and mobile navigation links with active indicators.
- `HealthStatusPill`: Real-time backend connectivity badge (`ONLINE • DB READY`).
- `NotificationBell`: Icon with numeric unread counter badge.
- `UserMenuPill`: Avatar circle with username, role tag, and dropdown.

2. Challenge & Solving:

- `ChallengeCard`: Points tag, difficulty badge, solve checkmark, live Docker indicator.
- `DifficultyBadge`: Color-coded pill (`Easy`: green, `Medium`: yellow, `Hard`: red, `Insane`: purple).
- `FlagSubmissionInput`: Monospace input field with submit button and instant animation.
- `LiveInstanceWidget`: Start/Stop button, status badge, host:port link, ticking countdown timer.
- `HintAccordion`: Title, point deduction warning, unlock confirmation dialog.
- `WriteupViewer`: Markdown-rendered solution with locked overlay state.

3. Leaderboard & Competitions:

- `PodiumCard`: Elevated cards for 1st (Gold), 2nd (Silver), and 3rd (Bronze) places.
- `RankingsTable`: Paginated table with rank numbers, handles, colleges, scores, and solve counts.
- `CountdownTimer`: Live ticking ticker (`XXd XXh XXm XXs`) with state transitions.

- `FrozenScoreboardBanner`: High-contrast alert bar indicating frozen public standings.

4. Profile & Teams:

- `ActivityHeatmap`: 52-week (364 days) grid with 5 level-intensity colors.
- `BadgeCard`: Icon, title, unlock criteria, award timestamp, locked/unlocked styling.
- `TeamRosterList`: Member avatars, captain crown indicator, invite code copy-button.

5. Admin & Moderation:

- `AnalyticsCard`: KPI number, trend percentage, and metric label.
- `SecurityIncidentRow`: Severity badge (`CRITICAL`/`HIGH`), incident type, review action button.
- `UserModerationRow`: Handle, email, role dropdown, ban toggle, mute toggle.

6. Feedback & Primitives:

- `ToastContainer`: Fixed alert notifications (success, error, warning, info).
- `ModalContainer`: Accessible backdrop modal with header, body, and action buttons.
- `SkeletonLoader`: Shimmering placeholder blocks for asynchronous queries.
- `EmptyState`: Clean icon, explanatory message, and action button.

---

21. Important User Flows

21.1 Student Journey

1. Registration & Auth: Registers with username, email, and password → logs in → stores JWT in `localStorage` → header updates.
2. Challenge Discovery: Browses challenge arena → filters by `Web` and `Medium` → selects challenge.
3. Attempt & Solving: Reads prompt → downloads attachment file → launches live Docker sandbox → inspects target → enters flag → receives instant success toast → solve recorded → points added to profile.
4. Hint Unlock: Encounters difficult challenge → views hint cost → confirms deduction dialog → hint revealed → penalty logged in backend.
5. Team Collaboration: Navigates to Teams → creates squad → shares invite code with classmates → members join.
6. Tournament Competition: Navigates to Events → registers team → waits for countdown ticker to reach zero → solves event challenges → monitors event scoreboard.
7. Certificate Claiming: Event concludes → student clicks "Get Digital Certificate" → receives cryptographic token → shares verification link.
8. Community Discussion: Opens challenge discussions → reads debrief comments → posts writeup tip.

21.2 Admin / Moderator Journey

1. Challenge Authoring: Opens Admin Console → Challenge tab → fills title, category, points, and flag → uploads attachment → publishes challenge.
2. Event Scheduling: Opens Events tab → sets start/end dates → assigns challenges with point overrides → freezes scoreboard in the final hour → unfreezes after closing → exports CSV results.
3. Anti-Cheat Triage: Opens Anti-Cheat tab → filters by `CRITICAL` → inspects duplicate flag submission incident → reviews IP collision log → changes status to `ACTIONED` with audit note.
4. User Moderation: Searches user handle → mutes user for forum spam → updates role from student to moderator.
5. Docker Infrastructure: Opens Live Sandboxes tab → views running containers → force-terminates orphaned instance → triggers cleanup sweep.

---

22. Required UI States

UI State	Backend Condition	Required Visual Representation
Loading / Fetching	Asynchronous API call in progress	Shimmering skeleton loader; disabled buttons with spinner
Empty State	API returns empty collection `[]`	Clean cybersecurity icon, descriptive empty text (e.g. "No challenges found in this category"), clear action button

UI State	Backend Condition	Required Visual Representation
Already Solved	`/status` returns `status: "solved"`	Green border, "SOLVED" checkmark badge, points earned tag, flag input pre-filled or disabled
Incorrect Flag	`/submissions` returns `is_correct: false`	Red shake animation on input, toast message: "Incorrect flag. Try again!"
Correct Flag	`/submissions` returns `is_correct: true`	Green pulse animation, celebratory toast with points awarded, solve counter increments
Writeup Locked	`/writeups` returns HTTP 403	Locked padlock icon, text: "Writeup locked until you solve this challenge"
Scoreboard Frozen	`is_scoreboard_frozen = true`	High-visibility warning banner: "Scoreboard is currently frozen for competition integrity"
Event Upcoming	`status = "UPCOMING"`	Live countdown ticker (`Starts in 02d 14h`), registration button enabled, challenges hidden
Event Ended	`status = "ENDED"`	"CONCLUDED" badge, final rankings, certificate claim button visible
Instance Starting	`status = "STARTING"`	Yellow pulse pill, "Container initializing...", spinner
Instance Running	`status = "RUNNING"`	Green pill, clickable link (`http://host:port`), ticking expiration timer, red Terminate button
Instance Expired	`status = "EXPIRED"`	Red tag, "Instance lease expired", Re-launch button
Account Muted	`is_muted = true`	Amber banner on forum: "Your account has been muted by a moderator", comment input disabled
Account Banned	`is_banned = true`	Red alert on login: "Account has been suspended. Contact administrator."

23. Role Permission Matrix

Platform Feature	Student	Moderator	Admin
Browse & Search Practice Challenges	✅	✅	✅
Download Challenge Attachments	✅	✅	✅
Submit Flags for Scoring	✅	✅	✅
Launch Live Docker Sandbox Instances	✅	✅	✅
Unlock Tiered Hints	✅	✅	✅
View Challenge Writeups (Post-Solve)	✅	✅	✅
View Unlocked Writeups Anytime	✅	✅	✅
Create, Edit, or Deactivate Challenges	❌	❌	✅
Upload Challenge Files & Flags	❌	❌	✅
Create & Join Teams (Max 4 Members)	✅	✅	✅
Register for CTF Events	✅	✅	✅
View Event Challenges (When Live)	✅	✅	✅
View Event Challenges (Anytime)	❌	❌	✅
Create & Schedule CTF Events	❌	❌	✅
Freeze & Unfreeze Event Scoreboards	❌	❌	✅
Export Event Results (JSON & CSV)	❌	❌	✅
Claim Verifiable Digital Certificate	✅	✅	✅
View Global & College Leaderboards	✅	✅	✅
Post Discussion Topics & Comments	✅ (if not muted)	✅	✅
Pin, Lock, or Delete Discussions	❌	✅	✅
Receive In-App Notifications	✅	✅	✅

Platform Feature	Student	Moderator	Admin
View Admin Analytics & Metrics	□	□	□
View Anti-Cheat Security Incidents	□	□	□
Triage & Review Security Incidents	□	□	□
Monitor All Active Docker Containers	□	□	□
Force-Kill Docker Sandboxes & Prune	□	□	□
Search & Moderate User Accounts	□	□	□
Promote User to Moderator	□	□	□
Promote User to Admin	□	□	□
Ban or Unban User Accounts	□	□	□

24. Stitch Frontend Design Context

24.1 Product Identity

The College OWASP CTF Platform is a modern, dark-first cybersecurity training ground and tournament hub. It bridges:

- Jeopardy CTF Arena (Hack The Box / CTFd competitive feel).
- Persistent Practice Arena (LeetCode / TryHackMe skill progression).
- Academic Club Identity (OWASP Student Chapter branding, college leaderboards, cryptographic digital certificates).

24.2 Visual Aesthetics & Color Palette

- Primary Background: Deep Obsidian / Slate (`#060911`).
- Surface & Cards: Elevated Midnight Navy (`#0b0f19` to `#0f172a`), with subtle border lines (`#1e293b`).
- Cyber Accents:
- Electric Cyan (`#06b6d4`, glow `rgba(6, 182, 212, 0.25)`): Primary actions, links, system status, ports.
- Cyber Purple (`#8b5cf6`, glow `rgba(139, 92, 246, 0.25)`): Secondary brand accent, insane difficulty, special badges.
- Emerald Green (`#10b981`): Solved status, correct flag feedback, success notifications.
- Amber Warning (`#f59e0b`): Hints, point deduction alerts, medium difficulty, active streaks.
- Crimson Red (`#ef4444`): Hard difficulty, anti-cheat critical alerts, force terminate, bans.
- Typography:
- Primary UI: `Inter` (clean, high legibility, 300 to 800 weights).
- Code, Flags, Hashes, IPs, Timers: `JetBrains Mono` (crisp monospace alignment).
- Styling Rules:
- Avoid generic flat templates; utilize subtle border glows, glassmorphic headers (`backdrop-filter: blur(12px)`), and micro-animations.
- Fully responsive across desktop (1440px), laptop (1024px), tablet (768px), and mobile (375px).

25. Backend Implementation Status

Phase	Functional Area	Architecture / Service	Status
Phase 1	Database Foundation & Core Models	PostgreSQL + SQLAlchemy 2.0 + Alembic (23 tables, 45 FKs, 145 indexes)	Fully Implemented
Phase 2	Authentication, JWT, and RBAC	`AuthService`, bcrypt hashing, JWT tokens, Student/Mod/Admin roles	Fully Implemented

Phase	Functional Area	Architecture / Service	Status
Phase 3	Challenge Catalog & File Attachments	`ChallengeService`, `StorageService`, safe file streaming, categories	Fully Implemented
Phase 4	Submissions & Authoritative Scoring	`SubmissionService`, `ScoringService`, solves ledger, duplicate protection	Fully Implemented
Phase 5	Leaderboards & Deterministic Ranks	`LeaderboardService`, global & college standings, composite indexes	Fully Implemented
Phase 6	Tiered Hints & Author Writeups	`HintService`, `WriteupService`, point penalties, post-solve gating	Fully Implemented
Phase 7	Teams & Collaborative Squads	`TeamService`, max 4 members, captaincy, transfer, invite codes	Fully Implemented
Phase 8	Events, Tournaments, & Freeze	`EventService`, countdowns, registrations, scoreboard freeze/unfreeze	Fully Implemented
Phase 9	Profiles, Streaks, & Activity Heatmap	`ProfileService`, `StreakService`, `ActivityService`, 365-day heatmap	Fully Implemented
Phase 10	Badges & Achievements Engine	`BadgeService`, automated rule evaluation, 8 seeded badges	Fully Implemented
Phase 11	Community Discussions & Comments	`DiscussionService`, challenge topics, nested replies, mute guards	Fully Implemented
Phase 12	In-App Notification Center	`NotificationService`, unread counts, batch read-all, event triggers	Fully Implemented
Phase 13	Admin Analytics Suite	`AnalyticsService`, 16 metric endpoints, distributions, trends	Fully Implemented
Phase 14	Anti-Cheat & Security Monitoring	`AntiCheatService`, rate limiting, rapid solve alerts, moderator reviews	Fully Implemented
Phase 15	Live Docker Container Sandboxes	`DockerChallengeService`, port leases, container TTL, cleanup sweeps	Fully Implemented
Phase 16	Security Hardening & Export/Certs	Security headers, CSP, HMAC certificates, CSV/JSON event export	Fully Implemented
Phase 17	Production Database & Unified Frontend	Supabase PostgreSQL migration head `0014`, Single-Page App	Fully Implemented

(All 233 automated test suites passing with 100% success rate).